

Aufgabenblatt 3

Backtracking & Greedy Algorithmen

Zur Erinnerung

- Melden Sie sich für die Prüfung auf MOSES an.
- Alle Hausaufgaben sind in Einzelarbeit zu erledigen. Kopieren Sie niemals Code und geben Sie Code in keiner Form weiter.
- Wenn Ihre Abgabe nicht im richtigen Ordner liegt, nicht kompiliert, unerlaubte packages oder imports enthält oder zu spät abgegeben wird, gibt es **0 Punkte** auf diese Abgabe.

Abgabe (bis 02.06.2025 23:59 Uhr)

Die folgenden Dateien müssen für eine erfolgreiche Abgabe im git Ordner eingecheckt sein:

Geforderte Dateien:

Blatt03/src/PermutationTest.java Aufgabe 4

Als Abgabe wird jeweils nur die letzte Version im main branch in git gewertet.

Aufgabe 1: Greedy Algorithmen (Tutorium)

Gehen Sie davon aus, dass Sie möglichst viele Apps auf Ihrem Handy installieren wollen und noch 1 GB Speicher nutzen können. Die einzige Information, die Sie zu den Apps haben ist der benötigte Speicherplatz.

- 1.1 Was wäre eine sinnvolle Greedy Auswahlstrategie um die größtmögliche Anzahl an Apps zu installieren? Gehen Sie an folgendem Beispiel durch, wie so ein Greedy Algorithmus vorgehen würde:

App1 (400MB), App2 (200MB), App3 (140MB), App4 (160MB), App5 (100MB), App6 (110)
In welcher Reihenfolge werden die Apps ausgewählt?

Lösung

Es werden sie Apps in aufsteigender Reihenfolge gewählt, gemessen an dem Speicherplatz, den sie benötigen: 100, 110, 140, 160, 200
Also App5, App6, App3, App4, App2 Damit wurden 5 Apps installiert, die insgesamt 710 MB Speicherplatz belegen. 290 MB sind also noch frei.

- 1.2 Wird mit dem Greedy Algorithmus der Speicherplatz optimal (ggf. vollständig) ausgenutzt?

Lösung

Nein. Im Beispiel oben: 710 MB, nicht 1 GB.
Es könnten auch die Apps App1, App2, App3, App4 und App5 gewählt werden, die $(400 + 200 + 140 + 160 + 100)\text{MB} = 1000\text{MB}$ Speicherplatz belegen und somit den verfügbaren Platz vollständig ausnutzen.

- 1.3 Wird mit einem Greedy Algorithmus die maximale Anzahl an Programmen installiert?

Lösung

Ja. Der Vollständigkeit halber hier der Beweis durch Widerspruch:
Annahme: Es wird nicht die maximale Anzahl an Programmen installiert.
Das heißt: Es gibt eine App A_i , die durch 2 Apps A_j und A_k ersetzt werden kann. Diese Apps sind vorher nicht durch den Greedy Algorithmus erfasst worden, also gilt für den benötigten Speicherplatz: $s_i \leq s_j \leq s_k$ (1). Da A_j und A_k Teil der optimalen Lösung sind, muss gelten $s_j + s_k < s_i + s_k$, dementsprechend $s_j < s_i$, dann hätte der Greedy Algorithmus aber diese App gewählt (widerspricht (1)).
Alternative Erklärung des Beweis durch Widerspruch:
Der benötigte Speicher einer App A_x wird im Folgenden mit s_x bezeichnet. Ausgangslage wie oben, das heißt, die App A_i ist die größte App, die noch installiert wurde. Außerdem gibt es einen restlichen verbleibenden Speicher v . Es gibt Apps, A_j und A_k , die nicht in der mit dem Greedy-Algorithmus gefundenen Lösungsmenge sind. Es muss also gelten, dass $s_i \leq s_j$ und $s_i \leq s_k$ (2). Andernfalls wäre A_j oder A_k in der Lösungsmenge. Außerdem muss gelten, dass $v < s_j$ und $v < s_k$. Wäre die nicht der Fall, wäre auf dem Handy noch Platz für eine der beiden Apps.

Wenn jetzt angenommen wird, dass der Algorithmus NICHT die größtmögliche Anzahl Apps herausgefunden hätte, dann müssten eine App (A_i) aus der Lösungsmenge getauscht werden gegen zwei Apps, die nicht in der Lösungsmenge sind (A_j und A_k). Damit durch die Installation dieser beiden Apps die Speicherkapazität nicht überschritten wird, muss gelten $s_j + s_k \leq s_i + v$. Wenn man nun auf beiden Seiten der Ungleichung v subtrahiert, so gilt $x + s_k \leq s_i$ (3) mit $x = s_j - v$. Dieses x muss eine positive Zahl sein, da $v < s_j$ (siehe oben). (3) steht im Widerspruch zu (2).

Aufgabe 2: Backtracking (Tutorium)

Das N Queen Problem beschreibt die Aufgabe N Damen auf ein $N \times N$ Schachfeld zusetzen, ohne dass sie sich gegenseitig schlagen können.

2.1 Wie würden sie Backtracking in diesem Fall anwenden?

Lösung

Die erste Dame auf das erste Feld setzen, die nächste auf das nächste, auf dem sie nicht geschlagen wird usw. bis es kein gültiges Feld mehr gibt. Dann die zuletzt gesetzte Dame auf das nächste mögliche Feld umsetzen. Starte mit leerer Lösung. Der Algorithmus wird im Vorlesungsvideo erklärt.

2.2 Führen sie für den Fall $N = 4$ eine Handsimulation durch.

Lösung

Q = von Dame besetztes Feld
x = nicht besetzbares Feld (von Damen bedroht)

Q	x	x	x
x	x		
x		x	
x			x

1.a

Q	x	x	x
x	x	x	Q
x	Q	x	x
x	x	x	x

1.a, 2.b, 1.a, Ende.

Keine andere Möglichkeit für 3. Zug, gehe zurück zu 2. Zug. Auch dort keine andere Möglichkeit, gehe zurück zu Zug 1.

Q	x	x	x
x	x	Q	x
x	x	x	x
x		x	x

1.a, 2.a, Ende

Q	x	x	x
x	x	x	Q
x		x	x
x	x		x

1.a, 2.b

x	Q	x	x
x	x	x	
	x		x
	x		

1.b

x	Q	x	x
x	x	x	Q
Q	x	x	x
x	x	Q	x

1.b, 2.a, 3.a, 4.a:

x	Q	x	x
x	x	x	Q
	x	x	x
	x		x

1.b, 2.a

x	Q	x	x
x	x	x	Q
Q	x	x	x
x	x		x

1.b, 2.a, 3.a

Aufgabe 3: JUnit Tests (Tutorium)

Betrachten Sie das folgende Programm, welches Sie auch im git finden:

```
public class StringGenome {
    private String s = "";

    public void addNucleotide(char c) {
        if (c == 'A' || c == 'C' || c == 'G' || c == 'T')
            s = s + c;
        else
            throw new RuntimeException("Illegal nucleotide");
    }

    public char nucleotideAt(int i) {
        if (i < s.length())
            return s.charAt(i);
        else
            throw new RuntimeException("Genome out of bounds");
    }

    public int length() {
        return s.length();
    }

    public String toString() {
        return s;
    }

    @Override
    public boolean equals(Object obj) {
        StringGenome i = (StringGenome) obj;
        return i.s.equals(this.s);
    }

    @Override
    public int hashCode() {
        return this.s.hashCode();
    }
}
```

3.1 Welche Aspekte der Klasse würden Sie in einem JUnit Test testen?

Lösung

Funktionalität der Methoden und Exceptions. Siehe Code.

3.2 Wie könnte man diesen Test implementieren? Welches sind die wichtigsten Befehle, die in JUnit-Tests benutzt werden?

Sie finden eine Testvorlage ebenfalls im git. Damit Sie JUnit nutzen können, müssen Sie ggf. JUnit5 zum classpath hinzufügen.

Lösung

Einführen: `assertEquals`, `assertTrue`, `assertFalse`, etc.
siehe Code

Aufgabe 4: Für die Hausaufgaben wird keine Lösung bereitgestellt.

Was Sie nach diesem Blatt wissen sollten:

- wie Greedy Algorithmen Entscheidungen treffen und welche Informationen dafür genutzt werden?
- was Rekursion und rekursive Aufrufe sind
- wie Backtracking funktioniert und wie Rekursion dabei genutzt wird
- für welche Probleme Backtracking geeignet ist
- was der Baum der Teillösungen ist und wie Backtracking und Greedy Algorithmen anhand dessen erklärt werden können
- was JUnit Tests sind und wie man diese schreibt
- welche Arten von Fehlern über JUnit Tests überprüft werden